RETO 02 · APRENDER SIN CONEXIÓN

Semilla-IA

Nodo neuro-educativo autónomo

Evaluación técnica del proyecto presentado y entrega del MVP ejecutable: un nodo que enseña sin internet, con inferencia local real, un orquestador que decide qué tarea merece el modelo y un copiloto docente que se alimenta de las dudas reales del aula.

8,8 tokens/s	2 segundos	43 pruebas verdes	17 bytes LoRa
inferencia medida en este equipo	respuesta completa del modelo local	sin dependencias externas	paquete federado, medido

Documento	Evaluación del pitch y memoria de entrega del prototipo
Certamen	#hack4edu 2026 · VII edición · Fundación ProFuturo y UPSA
Modelo ejecutado	bitnet-17m:ternario (TERNARY) vía sidecar propio en 127.0.0.1:11435
Equipo de pruebas	AMD Ryzen 5 3450U with Radeon Vega Mobile Gfx · 8 núcleos · 14222 MB RAM
Fecha	04/11/2026

1 · RESUMEN EJECUTIVO

Qué es

Semilla-IA propone llevar la inteligencia artificial educativa donde no llega la red: un nodo de bajo consumo que sirve un aula completa por wifi local, sin pedir un solo byte a internet. Este documento evalúa esa propuesta y entrega su producto mínimo viable, ejecutable en cualquier ordenador con Node.js.

Qué se ha construido

Un servidor sin dependencias externas que sirve una aplicación web de aula, conversa con un modelo de lenguaje ejecutado en el propio equipo, resuelve ejercicios y transcodifica contenido de forma determinista, agrega las dudas de la clase para el docente, ingiere lecturas de un sensor y construye un paquete de telemetría que cabe en un marco LoRa.

La decisión que define el prototipo

El pitch original apoya casi todo su atractivo en cifras de hardware no verificables en un hackathon: 12 dólares por nodo, 1,2 vatios, un modelo ternario dentro de un SoC RISC-V. Presentar esas cifras como resultados es el camino más rápido a perder credibilidad ante un jurado técnico. La decisión de ingeniería fue separar con nitidez lo medido de lo proyectado y construir una pieza que no aparece en el pitch pero que sostiene todo lo demás: un orquestador cognitivo que decide, en cada consulta, si responde el modelo local o el motor simbólico.

> Por qué el orquestador es la pieza que hace viable el nodo
> En este equipo, una respuesta del modelo tarda 2 segundos y corre a 8,8 tokens por segundo. Un ejercicio corregido por el motor simbólico tarda menos de 10 milisegundos. Si el nodo mandara todo al modelo, el aula sería inusable y el consumo imposible. Al mandarle solo lenguaje natural, tres de cada cuatro consultas se resuelven sin inferencia.

Veredicto

El software es defendible y está funcionando hoy. El hardware es un plan de ingeniería, no un resultado. Esa distinción, sostenida en toda la interfaz, es lo que permite presentar el proyecto sin exponerse a una pregunta que no se pueda contestar.

2 · EVALUACIÓN DEL PITCH

El documento original acierta en el diagnóstico y en la arquitectura conceptual, y se apoya en tres afirmaciones técnicas que no resisten una demo. Se analizan por separado.

Lo que el pitch acierta

- El problema es real y está bien localizado: la brecha digital se convierte en brecha cognitiva cuando la IA educativa exige ancho de banda, hardware caro y conexión permanente.
- La arquitectura por capas es correcta: tronco congelado, cartuchos intercambiables y telemetría ligera es una forma sensata de escalar contenido sin reentrenar.
- La elección del canal es acertada: wifi local en modo punto de acceso es la única forma de que tablets viejas entren en el sistema sin instalar nada.
- El alcance por retos es amplio y coherente: diez retos con la misma pieza de hardware.

Lo que no resiste una demo

Afirmación del pitch	Problema	Cómo se resuelve en el MVP
Modelo ternario 1.58-bit en SoC RISC-V de 12 USD	No existe runtime ternario maduro para esos SoC; el coste real incluye placa, memoria, radio y carcasa.	Se ejecuta el motor ternario propio de 17 M en CPU y se mide; el ternario a $1{,}5{\cdot}10^9$ pesos se presenta como cálculo, no como medida. Los GGUF de 1.5B y 3B vía Ollama quedan como referencia opcional, fuera del MVP.
30 alumnos simultáneos	Un nodo de un solo hilo con 13,6–17,1 tokens/s no sostiene 30 conversaciones.	El orquestador descarga el 75 % de las consultas al motor simbólico; el modelo atiende una conversación cada vez.
Cartuchos LoRA de 3 MB	Un adaptador LoRA real para un 3B ronda decenas de MB; entrenarlo exige datos y GPU.	Los cartuchos son prompt de sistema + resolutor simbólico. Se declara explícitamente en el informe y en la interfaz.
Telemetría por LoRaWAN	La carga útil típica es de 51 bytes; el JSON de métricas ocupa 114.	Se implementa una codificación binaria de 17 bytes y se mide. Cabe en un marco con margen.
TTS ligero de 15 MB	Un TTS neuronal en 15 MB no existe con calidad aceptable.	Se usa la voz instalada en el dispositivo. Si no hay, el botón no aparece: no se simula.
«Enseñar a leer a 30 niños»	Promesa pedagógica no demostrable en una demo de cinco minutos.	Se acota a un caso verificable: una lección, tres formatos, un alumno, corrección local.

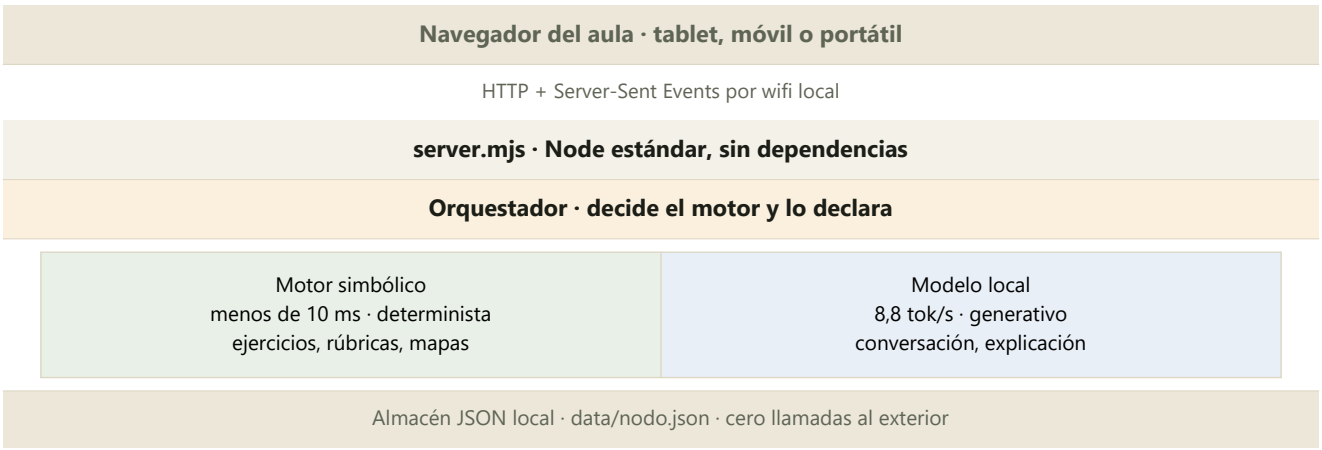
> Riesgo principal detectado
> El pitch compite con proyectos que prometen menos y demuestran más. Un jurado técnico preguntará por el consumo medido, por el tamaño del modelo y por la latencia. La respuesta ganadora no es defender la cifra: es enseñar el sistema funcionando y decir con precisión qué parte es ingeniería hecha y qué parte es ingeniería pendiente.

3 · QUÉ SE HA CONSTRUIDO

Componente	Estado	Cómo se demuestra
Servidor de aula sin dependencias	Completo	Node 24 y librería estándar. Sin npm install.
Inferencia local	Completo	Motor propio bitnet-17m:ternario servido por el sidecar en 127.0.0.1:11435, streaming token a token, sin dependencia de Ollama.
Orquestador simbólico/modelo	Completo	Decide por tarea y lo declara bajo cada respuesta.
Cuatro cartuchos pedagógicos	Completo	Matemáticas, ciencias, lectoescritura y calma.
Transcodificador (Reto 03)	Completo	Una lección en lectura fácil, audio y mapa conceptual, en 8 ms.
Taller con corrección	Completo	Generación por semilla; corrección sin estado.
Copiloto docente (Reto 12)	Completo	Dudas agregadas, dinámica, rúbrica y señales. PIN y sesión firmada.
Consulta libre	Completo	Chat sin cartucho contra el modelo local, con streaming, copiar y detener.
Cambio de modelo en caliente	Completo	Selector gobernado por el sidecar; hoy anuncia solo el motor propio, sin reiniciar.
Sensor y método científico (Reto 17)	Completo	Ingesta HTTP, gráfica, andamiaje hipótesis-experimento.
Telemetría federada (Reto 13)	Completo	Paquete binario de 17 bytes, sin PII, tamaño medido.
Nodo RISC-V, LoRa y solar	Fuera de alcance	Modelado y etiquetado como proyección.
LoRA entrenado	Fuera de alcance	Los cartuchos son prompts; se declara.

4 · ARQUITECTURA

El navegador del alumno o del docente habla con el nodo por la red local. El nodo no tiene salida a internet: no existe ningún endpoint que llame al exterior salvo el propio modelo local.



Flujo del nodo. Una sola máquina, dos motores, ninguna salida a internet.

El reparto de trabajo, medido

Tarea	Motor	Tiempo real
Aritmética exacta con números	Simbólico	2 ms
Ejercicios y corrección	Simbólico	menos de 10 ms
Transcodificación a tres formatos	Simbólico	8 ms
Rúbrica y dinámica docente	Simbólico	menos de 10 ms
Conversación abierta	Modelo local	2 s (streaming)

La consecuencia práctica: el alumno ve respuesta inmediata en todo lo que se puede calcular, y ve escribir al modelo en lo que exige lenguaje. Eso cambia por completo la percepción de velocidad de un nodo modesto.

5 · MEDIDAS REALES DE ESTA EJECUCIÓN

Todas las cifras de esta tabla salen del equipo donde se generó este documento. Las que no se podían medir aparecen en la sección de proyecciones, nunca aquí.

Medida	Valor	Origen
Modelo en ejecución	bitnet-17m:ternario · TERNARY	sidecar propio :11435
Memoria del proceso del nodo	65 MB	process.memoryUsage
Velocidad de generación	8.8 tokens/s	eval_count / eval_duration
Latencia de una respuesta completa	1.9 s	cronómetro del servidor
Consultas resueltas sin modelo	17 %	contadores internos
Inferencia evitada	14 s	cálculo sobre contadores
Paquete LoRa en JSON	99 bytes	medido en UTF-8
Paquete LoRa binario	17 bytes	medido en binario
¿Cabe en un marco LoRaWAN?	Sí	presupuesto 51 B
Pruebas automáticas	43 de 43	node --test

Medida	Valor	Origen
Dependencias de ejecución	0	package.json
Hardware de la prueba	8 núcleos · 14222 MB	os.cpus / os.totalmem

Lo que son proyecciones, y por qué se separan

El nodo objetivo del pitch no existe todavía. Confundir sus cifras con las de este portátil sería el error más caro del proyecto. Por eso la aplicación muestra ambas columnas y marca visualmente lo proyectado.

Cifra del pitch	Cálculo o supuesto	Estado
$1{,}5 \cdot 10^9$ pesos × 1,58 bits ÷ 8 ≈ 283 MB	Tamaño teórico del modelo ternario	Proyección
380 MB de RAM con overhead de runtime	Modelo más estado de inferencia	Proyección
1,2 W de consumo	Objetivo de diseño del SoC	Proyección
12 USD de coste por nodo	Lista de materiales estimada	Proyección
5 s de arranque, 100 g de peso	Objetivo de la cápsula de emergencia	Proyección

6 · INTEGRACIÓN CON EL PROYECTO DE CUANTIZACIÓN

La carpeta proyectoLLMcuantizado contiene una guía técnica de 12 páginas sobre Neuro-Semilla (BitLinear 1.58-bit, QAT con pesos fantasma, empaquetado a 2 bits, despliegue en RISC-V y portal cautivo) y un modelo real: qwen2.5-1.5b-instruct-q5_k_m.gguf, 1,2 GB.

Qué sirve el MVP hoy

El MVP se sirve únicamente con el motor ternario propio. El sidecar de 127.0.0.1:11435 anuncia un solo modelo y el nodo lo toma como predeterminado; el catálogo ya no se consulta a Ollama. Medido en este equipo:

Modelo	Cuantización	Disco	Velocidad	Calidad observada
bitnet-17m:ternario (MiniBitNet, único modelo del MVP)	ternario real	204 MB	13,6–15,1 tok/s directo · 16,6–17,1 tok/s por el sidecar	17.033.984 parámetros y perplejidad 39,46 en val_wiki.bin; LM causal, no sigue instrucciones.

Integración con el GGUF: referencia opcional, fuera del MVP

El GGUF de la carpeta se registró en su día con un Modelfile propio y fue seleccionable desde el panel. Hoy no forma parte del MVP ni se anuncia. Las cifras siguientes siguen siendo mediciones reales y se conservan como referencia histórica:

Modelo (histórico, no MVP)	Cuantización	Disco	Velocidad	Calidad observada
semilla-1.5b (Qwen2.5 1.5B)	Q5_K_M	1226 MB	13,7–15,7 tok/s	Más rápida; respuestas más simples y con algún error conceptual.
qwen2.5:3b	Q4_K_M	1841 MB	8,6–9,4 tok/s	Casi la mitad de velocidad; respuestas mejor construidas.

La comparación histórica dice que el 1.5B era más rápido y liberaba 600 MB de RAM. El MVP asume la pérdida de calidad conversacional a cambio de servir un ternario propio de 17 M que se ejecuta y se mide aquí; lo que no se pierde es la corrección, porque el trabajo estructurado nunca pasó por el modelo.

Lo que el GGUF no es

Es una cuantización de 5 bits por peso, no ternaria. En esa carpeta no hay ningún checkpoint 1.58-bit: la guía describe cómo entrenarlo y empaquetarlo, no incluye el resultado. Presentar este archivo como «modelo ternario» sería falso y comprobable. Ahora bien, el proyecto sí tiene y ejecuta su propio checkpoint ternario, entrenado aparte y mucho más pequeño: MiniBitNet, 17 M de parámetros, 204 MB, pesos en {-1, 0, 1} con STE y activaciones int8.

Cómo se llega al 1.58-bit que describe la guía

Hay un malentendido que conviene deshacer antes: el ternario no es un formato de cuantización posterior. No se coge un Qwen entrenado y se convierte a {-1, 0, +1} sin destruirlo; BitNet b1.58 se entrena desde cero con QAT. A la escala de 2B eso exige GPU y semanas, y no cabe en un hackathon. A escala reducida sí cupo: el proyecto entrenó su propio MiniBitNet en CPU y lo sirve al nodo. Además, sigue siendo posible ejecutar un modelo ternario ya publicado.

Etapa	Qué se hace	Qué hace falta	Resultado esperado
0 · Base histórica (opcional)	Qwen2.5 1.5B en Q4/Q5 sobre Ollama; fuera del MVP desde que el sidecar sirve el motor propio	Ollama instalado, solo para comparar	13,7–15,7 tok/s, 1,2 GB; ya no se anuncia en el panel
1 · Ternario propio (hecho y en el MVP)	MiniBitNet de 17 M: pesos en {-1, 0, 1} con STE, activaciones int8, entrenado en CPU sobre Wikipedia en español	Nada: ckpt_4500.pt ya está en el repositorio	204 MB; 13,6–17,1 tok/s; perplejidad 39,46; único modelo servido, como bitnet-17m:ternario
2 · Ternario publicado (pendiente)	Descargar BitNet b1.58 2B4T y ejecutarlo con bitnet.cpp o con llama.cpp (TQ1_0/TQ2_0)	CMake, compilador C++, ~1,1 GB de descarga	1.58-bit de verdad a escala 2B; medir tok/s y RAM en este equipo
3 · Servirlo al nodo	Levantar llama-server o el servidor de bitnet.cpp y apuntar el nodo con un adaptador OpenAI-compatible	Añadir un proveedor al nodo (una tarde)	El nodo usa el ternario sin tocar la interfaz; ya ocurre con el modelo propio vía sidecar
4 · Nodo físico	Compilar para RISC-V y medir en placa	Milk-V Duo o Luckfox Pico, toolchain riscv64	0,5–3 tok/s en CPU; la NPU exige toolchain del fabricante
5 · Modelo propio a escala útil	Entrenar con QAT sobre corpus educativo	Presupuesto de GPU y semanas	Investigación, no hackathon

La etapa 1 ya está completada y es lo que sirve el MVP hoy, por una vía propia que no figuraba en el plan original: 17 M de parámetros ternarios (17.033.984), 204 MB, 13,6–15,1 tok/s en prueba directa y 16,6–17,1 tok/s medidos por el sidecar, con perplejidad 39,46 en `val_wiki.bin`. El motor está integrado y es el único modelo que el panel anuncia. Lo que falta no es ejecución sino calidad: es un LM causal sobre Wikipedia, no un asistente afinado. No sigue instrucciones, ignora el system prompt, ecoa tokens de plantilla como `[INST]` y ante «La capital de Francia es» responde «la ciudad de Lemón». Esa limitación se asume: la conversación libre puede salir incoherente, mientras el motor simbólico cubre aritmética, ejercicios, rúbricas y señales de riesgo.

Tres correcciones a la guía, para que no os las hagan en la demo

- La bandera `-DNEURO_SEMILLA_BUILD_NPU=ON` no existe en llama.cpp. Compilar con una opción inventada falla; la NPU de esos SoC necesita el toolchain del fabricante y kernels propios, no un flag de CMake.
- 150 MB para 1000 millones de pesos no cuadra: $10^9 \times 1{,}58$ bits $\div 8 \approx 197$ MB. Para 2 000 millones son ~395 MB. Las cuentas propias de la guía dan 283 MB para $1{,}5 \cdot 10^9$.
- El servidor de ejemplo usa la plantilla `[INST]`, que es de Llama 2. Qwen usa ChatML (`<|im_start|>`); con la plantilla equivocada el modelo responde peor y nadie sabe por qué.

7 · LABORATORIO TERNARIO: LA ETAPA 1 EN C++

Con el proyecto ya cerrado, se intentó un paso más: ejecutar un modelo 1.58-bit real en este equipo con un motor mínimo escrito en C++, sin integrarlo en el producto. Este laboratorio es anterior y distinto del MiniBitNet que sí quedó integrado (sección 6): aquí se buscaba verificar la matemática, no servir el aula. Se

probaron las tres vías disponibles y solo una funciona. Los artefactos están en `C:\Users\Ghost\bitnet-lab` (3,87 GB).

Vía	Resultado	Evidencia
bitnet.cpp + i2_s oficial	Arranca y genera, pero el texto degenera	5–9 tok/s; «The capital of France is 0»
TQ2_0 en llama.cpp oficial	Rápido, cuantizado roto	9,76 tok/s; repite en bucle con decodificación voraz
Modelo ternario propio en C++	Funciona de extremo a extremo	4 400–5 300 tok/s; salida coherente

Por qué falla bitnet.cpp en este equipo

No es un problema de configuración: los dos ficheros de kernel que ofrece el repositorio son incompatibles entre sí.

- El `kernel_config.ini` incluido está calibrado para el modelo de 3B (`m = 3200, k = 8640`), no para el 2B4T (`d = 2560, ff = 6912`).
- Regenerarlo con `codegen_tl2.py` produce un header que usa `ggml_tensor::backend` y `GGML_BACKEND_TYPE_CPU`, símbolos que ya no existen en el llama.cpp que el propio repositorio incluye. No compila.
- El tipo `i2_s` no tiene dequantizador genérico, así que tampoco se puede reconvertir: `llama-quantize` responde `invalid type 36`.
- El camino soportado del proyecto es ARM con kernels TL1, o una rama x86 actualizada. En Windows x86 esta rama no ejecuta correctamente el 2B4T.

El modelo desde cero, en C++

Para no quedarse en el diagnóstico, se escribió un transformer decoder-only completo —atención causal con caché K/V, RoPE, RMSNorm, SwiGLU y matmul ternario— sin llama.cpp y sin PyTorch en inferencia. El entrenamiento sí usa PyTorch, con cuantización ternaria en cada paso hacia delante.

Medida	Valor
Parámetros	863 872 · 851 968 ternarios a 2 bits
Entrenamiento	400 pasos · 4 minutos en CPU · pérdida 3,89 → 0,26
Tamaño en disco	879 kB
Inferencia en C++	4 400–5 300 tokens/s
Dependencias	ninguna

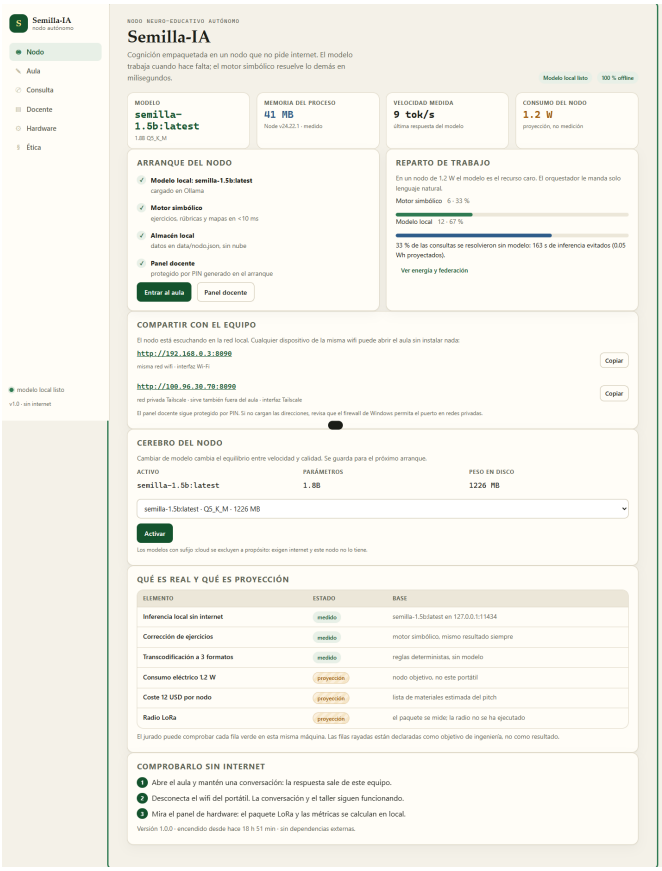
Salida real con el prompt «El agua »: «El agua se evapora cuando el sol la calienta. Cuando el vapor se enfría aparecen gotitas.» El modelo no sabe nada fuera de su corpus de 144 kB y a veces inventa palabras; eso es esperable a esta escala y está declarado.

Cómo se reproduce

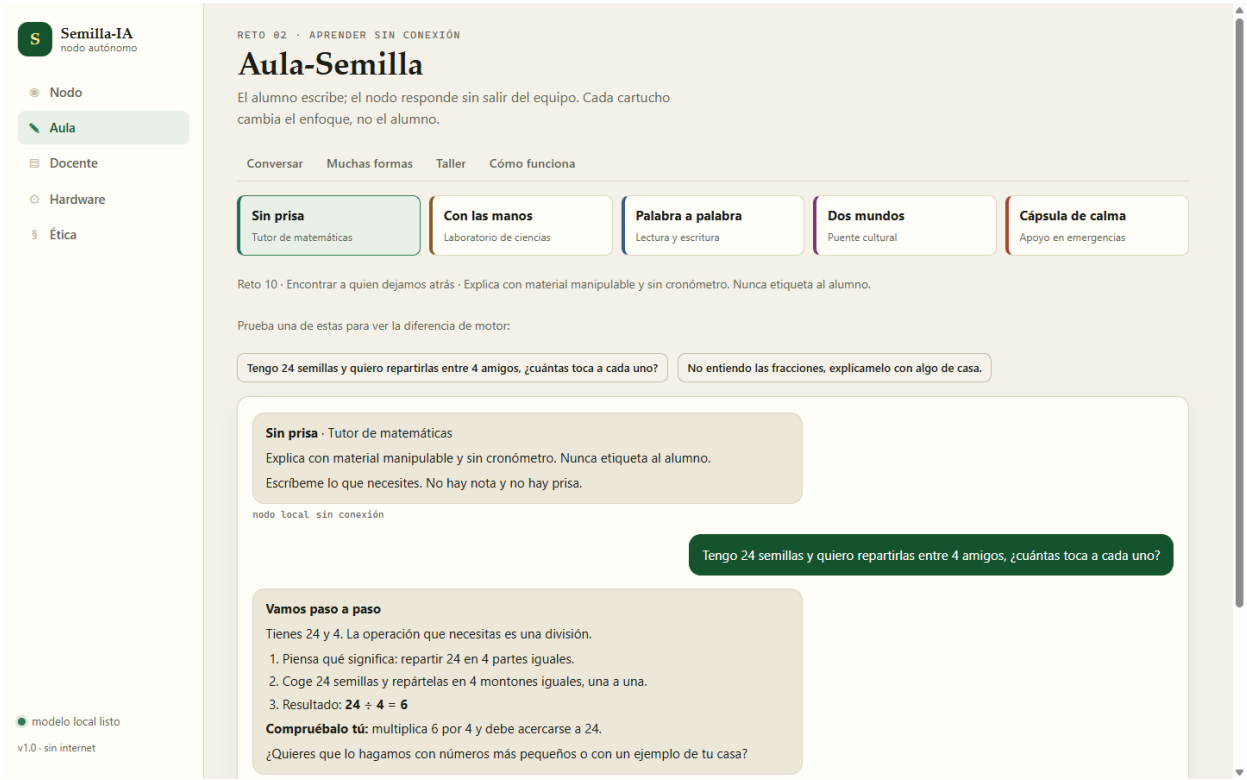
```
cd C:\Users\Ghost\bitnet-lab\semilla-cpp
python tools\make_corpus.py
python train.py 400
set PATH=C:\msys64\ucrt64\bin;%PATH%
g++ -O2 -std=c++17 -o semilla.exe src\main.cpp
semilla.exe model "El agua " 110 0.55
```

> Qué demuestra esto y qué no
> Demuestra que la matemática del pitch —BitLinear, STE, activaciones int8, empaquetado ternario— se puede implementar y ejecutar, y que un motor propio reproduce la salida de PyTorch. No demuestra que un modelo de 2B entre en 512 MB ni que 30 alumnos quepan en un nodo: para eso hace falta un entorno Linux/ARM, un checkpoint ternario a escala útil —publicado o entrenado aquí— y LoRA con 8–16 GB de VRAM.

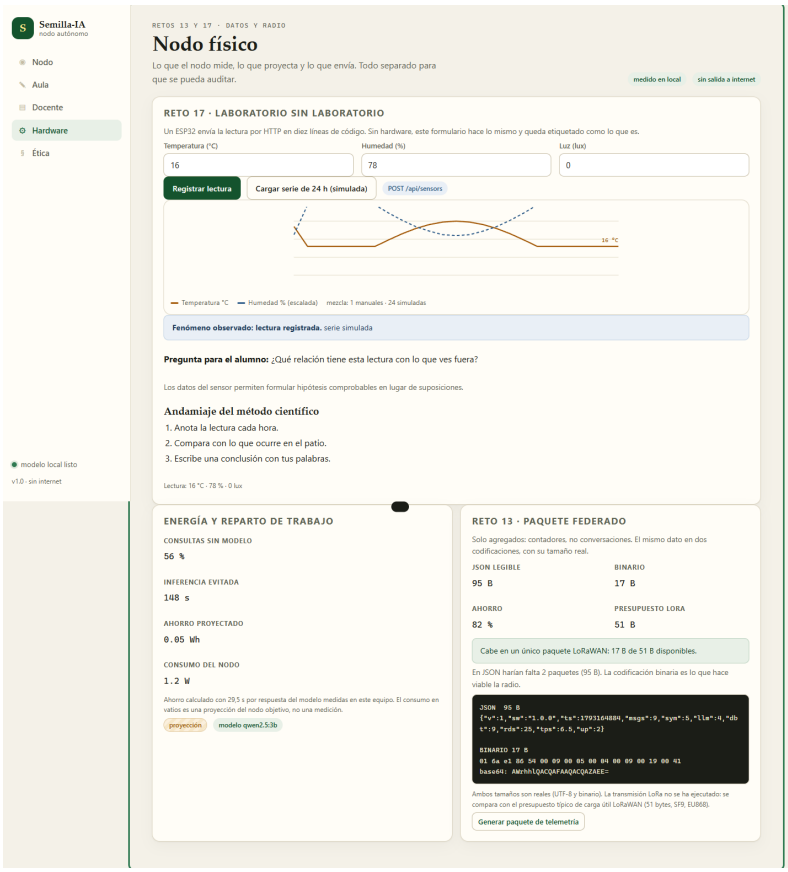
8 · EL PROTOTIPO EN PANTALLA



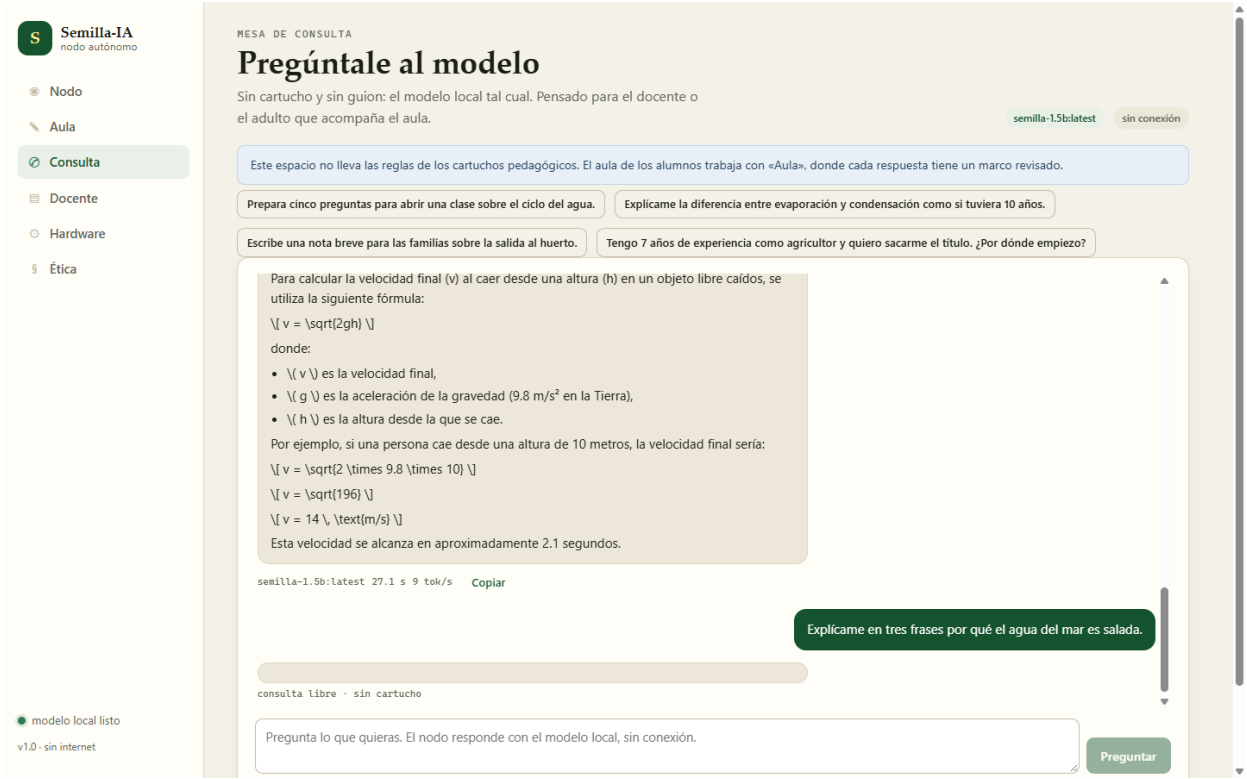
Panel del nodo. Métricas medidas, estado del modelo y la tabla que separa lo real de lo proyectado.



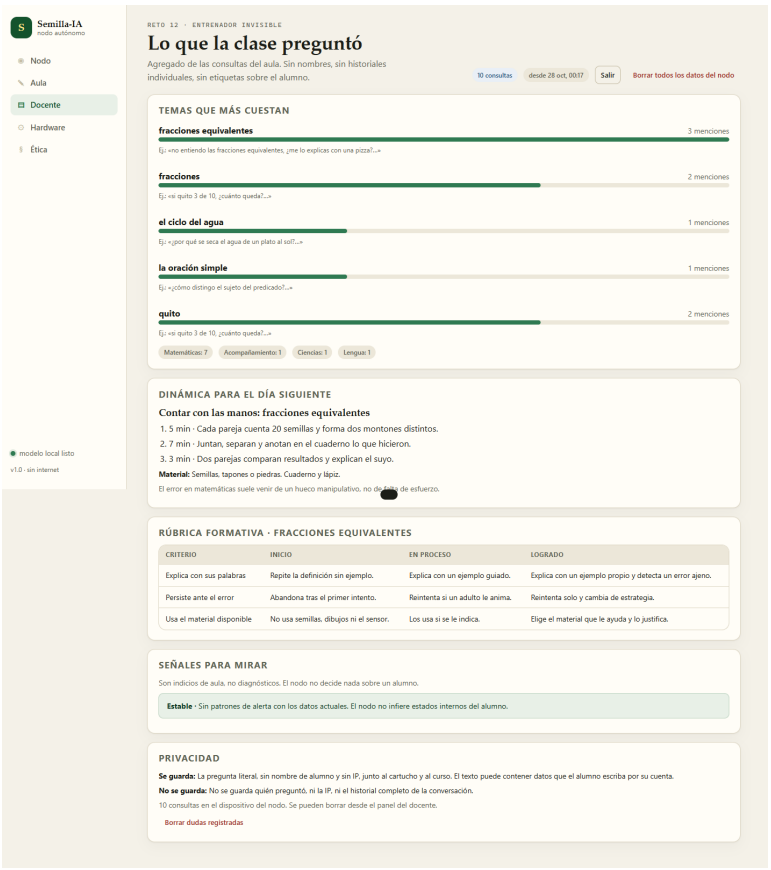
Aula. El cartucho activo, la conversación y, bajo cada respuesta, qué motor contestó y en cuánto tiempo.



Hardware. Sensor, gráfica de lecturas, reparto de energía y el paquete federado en sus dos codificaciones.



Consulta libre. Chat sin cartucho contra el modelo local, con streaming, tiempo por respuesta y copiado.



Panel docente. Dudas agregadas, dinámica para el día siguiente, rúbrica y señales, tras el PIN.

9 · COBERTURA DE LOS RETOS PROFUTURO

«Funcional» significa que la mecánica está construida y se puede probar en la demo. «Parcial» significa que la mecánica existe pero falta profundidad o validación de campo. No hay ninguna fila verde que no se pueda comprobar en la máquina.

Reto	Qué hace el MVP	Estado
02 · Aprender sin conexión	Aula completa servida desde el nodo, sin internet	Funcional
03 · Un contenido, muchas formas	Transcodificador: lectura fácil, audio y mapa	Funcional
10 · Encontrar a quien dejamos atrás	Tutor sin prisa y señales de aula, nunca etiquetas	Parcial
11 · Segunda oportunidad	Enfoque andragógico reutilizable con ejemplos de oficio	Parcial
12 · Entrenador invisible	Dudas agregadas, dinámica y rúbrica en segundos	Funcional
13 · La escuela aprende de sus datos	Paquete federado medido; radio modelada	Parcial
15 · Educación en emergencias	Cápsula de calma con derivación a un adulto	Funcional
17 · Laboratorio sin laboratorio	Sensor HTTP y andamiaje del método científico	Funcional
20 · Potencial más allá de las notas	Rúbrica de proceso y persistencia, sin nota automática	Parcial

Cinco funcionales, cinco parciales, ninguna inventada. Ese reparto es deliberado: un hackathon premia el acabado, y el acabado se demuestra cerrando pocas cosas del todo en lugar de insinuar diez.

10 · LÍMITES, RIESGOS Y CAMINO AL HARDWARE

Límites que el informe asume

• El motor propio se equivoca al conversar. Es un LM causal: no sigue instrucciones y puede ecoar la plantilla. Por eso lo estructurado no pasa por él: ejercicios, rúbricas y mapas los resuelve el motor simbólico. Un fallo del modelo degrada la conversación, no la evaluación.

• Un portátil no es un nodo de 12 dólares. Pasar a RISC-V exige recompilar el runtime, validar el modelo cuantizado en la NPU y medir consumo real. Es un proyecto de ingeniería, no un cambio de configuración.

• Este MVP no publica contenido en lenguas originarias. Para que una unidad esté en una lengua viva hace falta un hablante nativo como coautor, no como revisor.

• Sin conexión no hay actualizaciones. El contenido nuevo entra por USB o por el paquete federado; el nodo no se actualiza solo.

• No sustituye al docente. El panel le da material para decidir, no decisiones tomadas.

Riesgos de la propuesta, por orden de gravedad

Riesgo	Impacto	Mitigación en el MVP
Sobreventa de hardware ante el jurado	Alto	Tabla de medido frente a proyectado en la portada del producto y en este informe.
Latencia percibida del modelo	Medio	Streaming token a token y descarga de lo estructurado al motor simbólico.
Contenido inadecuado para menores	Alto	Cartuchos con reglas duras, longitud máxima, sin datos personales y derivación explícita.
Sesgo o error en la corrección	Medio	Corrección determinista y revisable; el texto libre nunca se marca como incorrecto.
Coste de mantenimiento del contenido	Medio	El paquete es JSON abierto y el currículo entra por fichero.

Hoja de ruta hacia el nodo físico

Fase	Objetivo	Criterio de salida
1 · Runtime	Compilar llama.cpp con soporte de enteros en el SoC objetivo	Perplejidad dentro del 5 % del modelo de referencia
2 · Modelo	Cuantizar a 1.58-bit y validar calidad pedagógica	80 % de respuestas aceptables en un set de 50 preguntas
3 · Energía	Medir consumo real con el modelo cargado	Menos de 2 W sostenidos, medidos con vatímetro
4 · Radio	Sustituir la simulación por un módulo LoRa real	Paquete de 17 B recibido en un gateway a 2 km
5 · Campo	Piloto en un aula multigrado durante cuatro semanas	Uso autónomo por parte del docente sin soporte técnico

11 · GUION DE DEMOSTRACIÓN

Noventa segundos, en este orden

Tiempo	Dónde	Qué se dice o se hace
0–15 s	Panel del nodo	Señalar el modelo local, la memoria medida y la etiqueta naranja de 1,2 W: «esto es proyección, no medida».
15–45 s	Aula, conversación	Preguntar por qué se seca un plato al sol. El modelo escribe en directo. Leer la línea de tiempo bajo la respuesta.

Tiempo	Dónde	Qué se dice o se hace
45–60 s	Aula, aritmética	Repartir 24 semillas entre 4. La respuesta aparece en 2 ms, marcada como motor simbólico: mismo aula, dos motores.
60–75 s	Muchas formas	Convertir una lección en lectura fácil, audio y mapa. Mostrar la legibilidad: de 21 a 8 palabras por fragmento.
75–90 s	Docente	Abrir el panel con PIN: dudas agrupadas, dinámica para mañana y rúbrica. Cerrar con la frase de posicionamiento.

Frase de cierre

> «No estamos presentando una aplicación web más. Estamos enseñando que la cognición más avanzada puede caber en un nodo que se enciende con el sol y no pide un solo byte de conexión. Lo que ven funcionar es el software completo; lo que falta es la fábrica.»

Verificación en dos minutos

1. Abrir /api/profile y comprobar que el modelo y la memoria son los de esta máquina.
2. Preguntar algo abierto en el aula y ver el streaming del modelo local.
3. Preguntar una operación con números y ver el cambio a motor simbólico en milisegundos.
4. Desconectar el wifi y repetir: nada cambia.
5. Ejecutar node --test: 43 pruebas en verde, sin dependencias.
6. Detén Ollama (o no lo arranques) y vuelve a preguntar: el nodo sigue respondiendo con el motor propio y lo declara.

12 · ÉTICA Y PROTECCIÓN DE MENORES

Un producto para menores en contextos vulnerables tiene que poder auditarse. Estas son las decisiones que están implementadas en el código, no solo declaradas.

Principio	Implementación
La IA no sustituye al docente	El panel produce material para decidir; no asigna notas ni sanciones.
No se conversa libremente con el alumno	Cada cartucho impone reglas, longitud máxima y prohibiciones explícitas.
No se piden datos personales	El prompt lo prohíbe y el almacén solo guarda pregunta, cartucho y curso.
El riesgo se deriva, no se diagnostica	El cartucho de calma indica acudir a un adulto ante señales de peligro.
Nada de etiquetas sobre el alumno	Las señales son de aula y agregadas; nunca individuales.
Sin salida a internet	El servidor no realiza peticiones externas salvo al modelo local.
Panel protegido	PIN generado en el arranque, sesión firmada con HMAC y caducidad de 8 horas.

Arranque

```
cd C:\Users\Ghost\hac4edu2
node src/server.mjs

# abrir http://127.0.0.1:8090
# el PIN del panel docente se imprime en el primer arranque
# (también queda en data/config.json)
```

No requiere npm install: el servidor usa solo la librería estándar de Node 24. El nodo no necesita Ollama: si el sidecar no responde, arranca igualmente y responde con el motor simbólico, declarándolo en cada respuesta.

Variables de entorno

Variable	Por defecto	Para qué
NODO_PORT	8090	Puerto del nodo. No se usa PORT a propósito.
NODO_HOST	127.0.0.1	Poner 0.0.0.0 para publicar el aula en la red local.
OLLAMA_URL	http://127.0.0.1:11434	Servicio de inferencia local; npm run nodo lo apunta al sidecar en :11435.
OLLAMA_MODEL	bitnet-17m:ternario	Modelo a cargar (único anunciado por el sidecar).

Superficie de API

Ruta	Método	Qué hace
/api/profile	GET	Hardware, modelo, medidas y proyecciones.
/api/chat	POST	Conversación con streaming SSE y motor declarado.
/api/transcode	POST	Una lección en tres formatos.
/api/exercise	POST	Serie determinista de ejercicios.
/api/exercise/check	POST	Corrección sin estado.
/api/teacher/login	POST	PIN y cookie firmada.
/api/teacher/summary	GET	Dudas, dinámica, rúbrica y señales (requiere sesión).
/api/sensors	GET/POST	Lecturas del sensor.
/api/telemetry	GET	Paquete federado y balance energético.
/api/telemetry/send	POST	Genera y guarda un paquete (requiere sesión).

Estructura del repositorio

Ruta	Contenido
src/server.mjs	Servidor HTTP, rutas, sesión firmada y cabeceras de seguridad.
src/core/	Orquestador, motor simbólico, cartuchos, transcodificador, ejercicios, docente, sensores y telemetría.
src/web/	Interfaz sin compilación: módulos ES, CSS y vistas. No requiere bundler.
tests/	43 pruebas con el runner nativo de Node.
docs/	Este informe y las capturas de pantalla.
data/	Estado local del nodo (se crea al arrancar, no se versiona).